Programação Concorrente: Linguagens e Técnicas

Concurrency

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Informatics,
NOVA School of Science and Technology

Rust: what are the guarantees

Features

- strict control of state mutation, via a type system
- shared memory (by default) immutable
- data-structures to manipulate shared memory need to be *unsafe code within safe APIs*

What one gets - if the program compiles:

- no memory violations occur
- thread-safety:
  shared memory accessed in *mutual exclusion*

Revisiting the Iterator Invalidator

In C++

```
std::vector<int> v { 10, 11 }; // Declares and initialises

int *vptr = &v[1]; // Points *into* ‘v’

v.push_back(12);// Inserts value 12 (what might go
wrong?)

std::cout « *vptr; // Bug (might use-after-free)
```

In Rust

```
let mut v = vec![10, 11]; // Declares and initialises

let vptr = &mut v[1]; // Points *into* ‘v’

v.push(12);// Inserts value 12 (what might go wrong?)

println!(“{}”, *vptr); // Compiler error: “cannot
borrow v as mutable more than once at a time”
```

A correct iterator

Ownership control: references are UNIQUELY owned

a reference is never aliased (i.e., shared) and mutable

```
fn add_something(v:&mut Vec<i32>) {
   v.push(11);
}
let mut v = vec![10];
add_something(&mut v);
v.push(11);
```

Ownership transfer

- call-by-value: makes a copy – controls *aliasing*
- call-by-reference: *borrowing, for a certain lifetime*

Concurrency

Aliased memory can only be shared and read

In particular, it can be accessed *concurrently*, i.e., "simultaneously"

It happens all the time in a computational device
(even in a mono-processor one)

Multitasking

Different (sub-)programs running simultaneously

Programming languages since the 80s have specific primitives for explicitly implement multitasking

A common abstraction: *threads*

"independent" portions of sequential code (aka *processes*)

Multithreading

Concurrent or parallel execution of threads

Threads in Rust

An executing Rust program is a collection of OS threads

There are two primitives to run code concurrently:

- `spawn`: unstructured – creates a new (detached) thread
- `join`: structured – may run in different thread

spawn

```
use std::thread;
let thread_join_handle = thread::spawn(move || {
  // some work here
});
// some work here
let res = thread_join_handle.join(); // join optional
```

More on threads in Rust

join is a parallel composition of closures: takes two closures and executes them in parallel, returning when both are done.

join is not in the Rust standard library – it is part of the Rayon crate

join

```
use std::thread;
thread::join(|| { // some work here } ...
             || { // some more work here });
// work here done when the code in the join terminates
```

Launches concurrently several pieces of sequential code

Example: concurrently reading a vector

```
let v = vec![10,11];

let vptr = &v[1];

join( || println!("v[1] = {}", *vptr),
```

Example with spawn (from Rust By Example)

```
use std::thread;
const NTHREADS: u32 = 10;

fn main() {
  // vector to hold the spawned children
  let mut children = vec![];

  for i in 0..NTHREADS {
    children.push(thread::spawn(move || {
      println!("this is thread number {}", i);
    }));
    }

  for child in children {
      let _ = child.join();
  }
}
```

Arc: Atomically Reference Counted

Useful to get shared ownership between threads

Thread Safety

Unlike Rc<T>, Arc<T> uses atomic operations for its reference counting

Example

```
let arc_v1 = Arc::new(vec![10, 11]);

let arc_v2 = Arc::clone(&arc_v1);

spawn(move || println!(“{:?}”, arc_v2[1]));

      println!(“{:?}”, arc_v1[1]);
```

How to safely share mutable references?

There are memory and thread safe APIs

Mutex (Mutual EXclusion access to *data*)

```
let mutex_v = Mutex::new(vec![10, 11]);

join(

   {let mut v = mutex_v.lock().unwrap(); v.push(12); },

   {let mut v = mutex_v.lock().unwrap();
    println!("{:?}", *v);});
```

- block threads waiting for the lock to become available
- *Lock* released when `v` dropped
- The program prints:

  [10, 11, 12] if the first *thread* acquires first the *lock*, or

  [10, 11] otherwise

RwLock - distinguish between readers and writers

Alternative to Arc.

Two ways of locking

- one to grant read-only access to any number of concurrent readers
- another one for exclusive write access